

Java History Questions - Cheat Sheet

Origin and early history

Q: Who created Java and when?

James Gosling and his team at Sun Microsystems. The project started in 1991 as “Oak”, originally targeting interactive TV and set-top boxes. Java 1.0 shipped publicly in January 1996.

Q: Why was Java created?

The original goal was a platform-independent language for consumer electronics. The team wanted something simpler than C++, more secure, and able to run on any device without recompilation. When interactive TV didn't take off, Java pivoted to the web (applets) and then to general-purpose development.

Q: Why is it called Java?

The team brainstormed names after “Oak” turned out to be already trademarked. “Java” came from the coffee they drank, specifically Java coffee from Indonesia. The other shortlisted name was “Silk”.

Q: What does “write once, run anywhere” mean?

Java source compiles to bytecode (`.class` files), which runs on any JVM. The JVM does the platform-specific work. So one compiled JAR runs on Windows, Linux, Mac, or any JVM-supported platform without recompilation.

Q: Who owns Java now?

Oracle. Sun Microsystems was acquired by Oracle in January 2010 for \$7.4 billion. Java came with the deal.

Q: When did Java become open source?

November 2006. Sun open-sourced the JDK under GPL, creating OpenJDK. The full source was released in stages through 2007-2008. Before that, Java had a reference implementation that wasn't fully open.

Java vs JavaScript

Q: Are Java and JavaScript related?

No. Classic interview gotcha. JavaScript was created by Brendan Eich at Netscape in 1995. It was originally called “Mocha”, then “LiveScript”, and renamed to “JavaScript” as a marketing tie-in to ride Java's popularity. The languages share C-like syntax and almost nothing else.

JDK, JRE, JVM, HotSpot

Q: What's the difference between JDK, JRE, and JVM?

- **JVM (Java Virtual Machine):** Runs bytecode. Platform-specific implementation of the JVM spec.
- **JRE (Java Runtime Environment):** JVM + standard libraries. Enough to run a Java app.
- **JDK (Java Development Kit):** JRE + compiler (`javac`) + tools (`javadoc` , `jar` , etc.). What developers install.

Since Java 9, the JRE is no longer shipped as a standalone download. The JDK includes everything, and you use `jlink` to build minimal runtime images.

Q: What's HotSpot?

The reference JVM that Sun (now Oracle) ships. Introduced in 1999, became the default JVM in Java 1.3 (2000). Replaced the older "Classic VM". Named for its adaptive optimization, which spends extra effort on the "hot" code paths.

Q: What's the JIT compiler?

Just-In-Time compiler. The JVM starts by interpreting bytecode, profiles which methods get called often, and compiles those to native machine code on the fly. That's why Java code often gets faster after running for a while.

Q: What's tiered compilation?

HotSpot ships two JIT compilers:

- **C1 (client compiler):** Compiles quickly with light optimization. Used for warmup.
- **C2 (server compiler):** Compiles slowly with aggressive optimization. Used for hot code.

Since Java 8, both run together. Bytecode starts interpreted, gets compiled by C1, then promoted to C2 if it stays hot.

Q: What's the difference between OpenJDK and Oracle JDK?

- **OpenJDK:** The open-source reference implementation. GPL-licensed. Maintained by Oracle and the community.
- **Oracle JDK:** Oracle's commercial distribution. Effectively identical to OpenJDK since Java 11, but with a different license for commercial production use.

Other distributions exist: Amazon Corretto, Azul Zulu, Eclipse Temurin (formerly AdoptOpenJDK), Microsoft Build of OpenJDK, GraalVM. All built from OpenJDK with their own build pipelines and support contracts.

Open source and the Java community process

Q: What's the JCP?

Java Community Process. The formal mechanism for evolving Java, established in 1998. Companies and individuals submit proposals (JSRs) that get reviewed by an Executive Committee. JCP gave Java a structured way to standardize without depending entirely on Sun's whims.

Q: What's a JSR?

Java Specification Request. A formal proposal for a Java technology. Examples: JSR 14 (Generics), JSR 175 (Annotations), JSR 310 (Date and Time API), JSR 376 (Module System).

Q: What's a JEP?

JDK Enhancement Proposal. A lighter-weight process introduced in 2011 for changes to the JDK itself (as opposed to the language spec). JEPs sit upstream of JSRs. Examples: JEP 286 (`var` for locals), JEP 425 (Virtual Threads).

Q: What's the TCK?

Technology Compatibility Kit. The test suite a JVM implementation must pass to call itself "Java compatible". This is the bar Amazon, Azul, and others have to clear for their distributions.

Version timeline

Q: What are the major Java versions and what did each add?

Version	Year	Headline features
1.0	1996	Initial release
1.1	1997	Inner classes, JDBC, RMI, JavaBeans
1.2 (Java 2)	1998	Collections framework, Swing, JIT compiler
1.3	2000	HotSpot JVM, JNDI
1.4	2002	<code>assert</code> , regex, NIO, XML APIs
5	2004	Generics, annotations, enums, autoboxing, for-each, varargs, new memory model
6	2006	Performance work, scripting engine, JDBC 4.0
7	2011	try-with-resources, diamond operator, switch on <code>String</code> , NIO.2
8 (LTS)	2014	Lambdas, streams, <code>Optional</code> , <code>java.time</code> , default methods
9	2017	Module system (Jigsaw), JShell
10	2018	<code>var</code> for local variables
11 (LTS)	2018	HTTP client API, new <code>String</code> methods, Java EE modules removed
12-13	2019	Switch expressions (preview), text blocks (preview)
14	2020	Switch expressions (final), records (preview)
15	2020	Text blocks (final), sealed classes (preview)
16	2021	Records (final), pattern matching for instanceof
17 (LTS)	2021	Sealed classes (final), pattern matching
18-20	2022-23	Virtual threads (preview), pattern matching for switch (preview)
21 (LTS)	2023	Virtual threads (final), sequenced collections, pattern matching for switch (final)
22	2024	Foreign Function & Memory API (final), unnamed variables
23	2024	Markdown javadoc, ZGC generational mode by default

Version	Year	Headline features
24	2025	Stream gatherers (final), structured concurrency progress
25 (LTS)	2025	Latest LTS release

Q: What's an LTS release?

Long Term Support. LTS versions get extended security and maintenance updates. Non-LTS releases are supported only until the next non-LTS release ships. LTS lineup so far: 8, 11, 17, 21, 25. Most production code targets one of these.

Q: Why did Java 2 get rebranded as Java 5?

Pre-5, versions were named like “Java 2 Platform Standard Edition 1.4”. Confusing for everyone. In 2004, Sun simplified branding: the spec version became the public version. So “1.5” became “Java 5”.

Q: Why did the release cadence change?

Before Java 9, major releases were years apart and unpredictable. Java 7 took 5 years after Java 6. Starting with Java 9 (2017), Oracle moved to a strict 6-month cadence: every March and September. LTS releases happen every 2 years now (since Java 17 in 2021).

Major project codenames

Q: What are the major Java project codenames?

Project	Goal	Delivered in
Coin	Small language changes (try-with-resources, diamond)	Java 7
Lambda	Closures, functional interfaces, streams	Java 8
Jigsaw	Module system	Java 9
Amber	Language productivity (records, sealed classes, text blocks, pattern matching)	Java 14-21
Loom	Virtual threads, structured concurrency	Java 21
Panama	Native interop (Foreign Function API, Vector API)	Java 22
Valhalla	Value types, generic specialization	In progress
Leyden	Faster startup, AOT compilation	In progress

Q: Why was Project Jigsaw controversial?

Modules changed how Java code is packaged and how access control works. A lot of existing code relied on reflection into internal JDK packages (especially `sun.misc.Unsafe`). Jigsaw closed those backdoors and broke libraries like Spring, Hibernate, and Lombok until they updated. Migration from Java 8 to 9+ was painful, and many projects stayed on 8 for years.

Garbage collection evolution

Q: What garbage collectors does Java ship?

GC	Introduced	Default in	Status
Serial GC	1.x	Small heaps	Still ships, niche use
Parallel GC	1.4	Java 5-8	Default for throughput
CMS (Concurrent Mark Sweep)	1.4	Never default	Deprecated in 9, removed in 14
G1 (Garbage First)	Java 7	Java 9+	Default for general workloads
ZGC	Java 11	Optional	Sub-millisecond pauses, large heaps
Shenandoah	Java 12	Optional	Low-latency, Red Hat-led

Q: Why was CMS removed?

Too much code, too many tuning knobs, and G1 covered most of the same use cases with less complexity. Deprecated in Java 9, removed in Java 14.

Q: What changed about ZGC recently?

ZGC went generational in Java 21 (preview) and became the default ZGC mode in Java 23. Generational ZGC handles short-lived objects much more efficiently than the original single-generation design.

Editions and splits

Q: What were J2SE, J2EE, and J2ME?

The three editions of Java 2:

- **J2SE (Standard Edition):** Desktop and server apps. The core of what most developers use.
- **J2EE (Enterprise Edition):** Servlets, JSP, EJB, JPA, JMS. Server-side enterprise stack.
- **J2ME (Micro Edition):** Mobile and embedded devices.

The “2” was dropped in Java 5: SE, EE, ME.

Q: What happened to Java EE?

Oracle handed Java EE to the Eclipse Foundation in 2017. It was renamed to Jakarta EE because Oracle kept the “Java” trademark. The `javax.*` packages were renamed to `jakarta.*` starting in Jakarta EE 9 (2020). That rename broke a lot of existing code and triggered a long migration cycle that’s still ongoing in some shops.

Q: What happened to Java ME?

Mostly dead. The mobile market moved to Android and iOS, both of which don’t use Java ME. Oracle still maintains Java ME Embedded for IoT use cases, but the broader ME story is over.

Notable deprecations and removals

Q: What were applets and what happened to them?

Small Java programs that ran inside a web browser via a plugin. Big in the late 90s and 2000s. Browser vendors dropped NPAPI plugin support around 2015-2017, breaking applets in practice. Java deprecated the Applet API in Java 9 and removed it in Java 17.

Q: Why was `finalize()` deprecated?

`Object.finalize()` was meant to be a destructor-like hook the GC would call before reclaiming an object. It was unreliable (no guarantee when or if it runs), slow, and a common source of resource leaks. Deprecated in Java 9, removed for removal in Java 18. Use try-with-resources and `Cleaner` instead.

Q: What happened to Nashorn?

The JavaScript engine that shipped with Java 8 (replacing Rhino). Deprecated in Java 11, removed in Java 15. GraalVM JavaScript is the modern replacement.

Q: What’s `sun.misc.Unsafe` and why is it controversial?

An internal Sun/Oracle class that gives direct memory access, bypassing Java’s safety guarantees. Never part of the public API, but widely used by libraries (Netty, Cassandra, Hadoop) for performance. Java 9 modules tried to hide it. Java 23 began deprecating it for removal. Replacements: `VarHandle`, the Foreign Memory API, and `MemorySegment`.

Q: What other notable things were removed?

- Java Web Start removed in Java 11.
- JavaFX unbundled from the JDK in Java 11, now a separate library.
- CORBA, JAX-WS, JAXB modules removed in Java 11.
- Pack200 removed in Java 14.
- Security Manager deprecated in Java 17, removal pending.

Important design decisions

Q: Why doesn’t Java support multiple inheritance?

To avoid the diamond problem, where a class inherits the same method from two different parents and there's no clear winner. Java allows multiple interface implementation, which sidesteps the issue because interfaces (originally) had no bodies. Default methods in Java 8 brought a controlled form of multiple inheritance for behavior, with a tiebreaker rule when interfaces conflict.

Q: Why doesn't Java have pointers?

By design. Pointers in C/C++ were a major source of security holes and crashes. Java uses references with no pointer arithmetic. The JVM manages memory through garbage collection, so manual `free()` calls aren't needed.

Q: Why is String immutable?

Four reasons: string pool caching, free thread safety, hashCode caching, and security. (See the String Immutability cheat sheet for the full story.)

Q: When did generics arrive, and why so late?

Java 5 (2004), 9 years after the language launched. Generics were retrofitted using type erasure to keep backward compatibility with pre-5 code. The cost: the runtime can't see generic type parameters. `List<String>` and `List<Integer>` look identical to the JVM after compilation.

Q: What was wrong with the original Java Memory Model, and when was it fixed?

The pre-Java 5 memory model was vague, contradictory, and made `volatile` and `synchronized` semantics hard to reason about. JSR 133 rewrote the memory model in Java 5 (2004). The new model formally defined happens-before relationships, fixed `volatile` (now provides ordering guarantees), and made double-checked locking actually work.

Q: Why is `main` static?

When the JVM starts a program, no objects exist yet. A static method belongs to the class itself, so the JVM can invoke it without instantiating anything. That's why the entry point is `public static void main(String[] args)`. Java 21 introduced unnamed instance main methods as a preview, which loosens this for simple programs.

Common trick questions

Q: What's the difference between `final`, `finally`, and `finalize`?

3 unrelated things that share a prefix.

```

// final - constant value or non-extendable class/method
final int MAX = 100;
final class Cannot {}

// finally - always runs after try/catch
try {
    risky();
} finally {
    cleanup();
}

// finalize - deprecated GC hook (don't use)
@Override
protected void finalize() {
    // called before GC reclaims the object
}

```

Q: Is Java pass-by-value or pass-by-reference?

Pass-by-value, always. The confusion comes from objects: Java passes the reference by value, which means the method gets a copy of the reference. Reassigning that copy doesn't affect the caller, but mutating the object it points to does.

```

void modify(int n) {
    n = 100; // doesn't affect caller
}

void mutate(List<Integer> list) {
    list.add(100); // affects caller (same object)
    list = new ArrayList<>(); // doesn't affect caller (local copy)
}

```

Q: What's the magic number at the start of every `.class` file?

`0xCAFEBADE`. The first 4 bytes of any valid Java class file. The next 2 bytes are the minor version, then 2 bytes for the major version (52 = Java 8, 55 = Java 11, 65 = Java 21).

Q: What's the difference between `==` and `.equals()` for objects?

`==` checks if two references point to the same object. `.equals()` checks content equality (when the class overrides it correctly). For primitives, only `==` exists and compares values directly.

Q: Can `main` be overloaded?

Yes, you can declare multiple `main` methods with different signatures. The JVM only calls the canonical `public static void main(String[] args)` to start the program. The rest are just regular methods.

Q: What's the size of `int` in Java?

Always 32 bits, regardless of platform. Java spec pins the sizes: `byte` 8, `short` 16, `int` 32, `long` 64, `float` 32, `double` 64. `char` is 16-bit unsigned. `boolean` is implementation-defined but

logically 1 bit.

Q: What was PermGen, and what replaced it?

PermGen was a separate memory region for class metadata and interned strings. It had a fixed size and caused `OutOfMemoryError: PermGen space` when class loaders piled up (common in web apps with hot-reload). Replaced by Metaspace in Java 8, which grows dynamically and lives in native memory.

JVM languages

Q: What other languages run on the JVM?

Language	First release	Origin
Java	1996	Sun Microsystems
Groovy	2003	James Strachan
Scala	2004	EPFL (Martin Odersky)
Clojure	2007	Rich Hickey
Kotlin	2011	JetBrains

All compile to JVM bytecode and interoperate with Java libraries. Kotlin became Google's preferred language for Android in 2017. Scala powered Twitter and LinkedIn back-ends for years. Clojure runs a lot of fintech systems.

Famous moments and controversies

Q: What was the Oracle vs Google lawsuit about?

Oracle sued Google in 2010, claiming Android's use of Java APIs violated copyright. The case bounced through courts for 11 years. In 2021, the US Supreme Court ruled in Google's favor: reimplementing API method signatures counts as fair use. The case had huge implications for whether APIs are copyrightable.

Q: What was the Java 11 licensing drama?

In 2018, Oracle changed the Oracle JDK license. The free LTS support window was cut from 5 years to 6 months for non-paying users. Cue panic and migration. Most teams moved to OpenJDK builds (Adoptium / Temurin, Corretto, Zulu). Oracle reversed course in 2021 with the No-Fee Terms and Conditions license, making Oracle JDK free again for production use.

Q: Why did Android stop being purely Java?

Google announced Kotlin as a first-class Android language in 2017 and as the preferred language in 2019. The shift was partly about modernization, partly about the Oracle lawsuit baggage. New Android

Google samples and APIs are now Kotlin-first.

Q: What happened to JavaFX?

Shipped with the JDK from Java 8 to Java 10. Unbundled in Java 11 and handed to the community (Gluon). Still alive as a separate OpenJFX project, mostly for desktop apps that need richer UI than Swing.

Modern era

Q: What are virtual threads (Project Loom)?

Java 21 (LTS, 2023) finalized virtual threads. They're lightweight threads scheduled by the JVM instead of the OS. Millions of virtual threads can run on a handful of OS threads. Designed for blocking I/O workloads like servers handling many concurrent connections. Started as Project Loom around 2018.

Q: What's Project Panama?

Better interop between Java and native code. Replaces the painful JNI with the Foreign Function & Memory API. Finalized in Java 22. Also includes the Vector API for SIMD operations.

Q: What's Project Valhalla?

Value types and primitive classes. Aims to close the gap between objects (heap-allocated, reference semantics) and primitives (stack-allocated, value semantics). Generic specialization is part of the goal. Still in development.

Q: What's GraalVM?

A polyglot JVM and high-performance JIT compiler. Also supports ahead-of-time (AOT) compilation to native binaries, which removes JVM startup cost and lowers memory usage. Common in serverless and CLI tool contexts. Spring Native and Quarkus build on GraalVM.

Q: What are records?

Compact syntax for data carriers. Finalized in Java 16.

```
public record Point(int x, int y) {}
```

That one line gives you a final class with `x()` and `y()` accessors, `equals`, `hashCode`, `toString`, and a canonical constructor.

Q: What are sealed classes?

A way to restrict which classes can extend or implement a type. Finalized in Java 17.

```
public sealed interface Shape permits Circle, Square, Triangle {}
```

Useful for closed type hierarchies and exhaustive pattern matching.

Quick reference

Fact	Answer
Original creator	James Gosling, Sun Microsystems
First release	Java 1.0, January 1996
Original name	Oak
Naming inspiration	Java coffee from Indonesia
Current owner	Oracle (acquired Sun in 2010)
Open source year	2006 (OpenJDK)
LTS versions	8, 11, 17, 21, 25
Latest LTS	Java 25 (September 2025)
Java EE renamed to	Jakarta EE (2017, Eclipse Foundation)
Generics added in	Java 5 (2004)
Lambdas and streams added in	Java 8 (2014)
Modules added in	Java 9 (2017)
Records finalized in	Java 16 (2021)
Sealed classes finalized in	Java 17 (2021)
Virtual threads finalized in	Java 21 (2023)
Foreign Function API finalized in	Java 22 (2024)
Class file magic number	0xCAFEBAFE
Release cadence	6 months, with LTS every 2 years
Default GC	G1 (since Java 9)
Memory model rewritten in	Java 5 (JSR 133)
PermGen replaced by	Metaspace (Java 8)
Default JVM	HotSpot